

```

/*
=====
=====
    LEDGER – Study consistency tracker
    Talks directly to Firebase
    Realtime Database via its REST API.
    No build step, no auth – designed
    to run as a static site.

=====
===== */

const DB_URL = "https://devpandaxd-
default-rtdb.asia-
southeast1.firebaseio.com";

// ----- In-memory state
(mirrors the DB) -----
let state = {
  blocks: {},          // id -> {date,
start, end, subjectId, subjectName,
topic, notes, createdAt}
  subjects: {},        // id -> {name,
topics: {id: {name, completed,
createdAt}}}
  practice: {}         // id -> {date,

```

```

subjectId, subjectName, attempted,
correct, createdAt}
};

let chart = null;
let pendingReturn = null; // { view:
'block'|'practice', data: {...} } –
set when subject is quick-added mid-
form

// ----- Firebase REST helpers -
-----
async function fbGet(path) {
  const res = await
fetch(`${DB_URL}/${path}.json`);
  if (!res.ok) throw new Error(`GET
${path} failed (${res.status})`);
  return res.json();
}
async function fbPost(path, data) {
  const res = await
fetch(`${DB_URL}/${path}.json`, {
    method: "POST",
    headers: { "Content-Type":
"application/json" },
    body: JSON.stringify(data)
  });
  if (!res.ok) throw new Error(`POST
${path} failed (${res.status})`);

```

```
    return res.json(); // { name: "<new
id>" }
}
async function fbPut(path, data) {
    const res = await
fetch(`${DB_URL}/${path}.json`, {
    method: "PUT",
    headers: { "Content-Type":
"application/json" },
    body: JSON.stringify(data)
});
    if (!res.ok) throw new Error(`PUT
${path} failed (${res.status})`);
    return res.json();
}
async function fbPatch(path, data) {
    const res = await
fetch(`${DB_URL}/${path}.json`, {
    method: "PATCH",
    headers: { "Content-Type":
"application/json" },
    body: JSON.stringify(data)
});
    if (!res.ok) throw new Error(`PATCH
${path} failed (${res.status})`);
    return res.json();
}
async function fbDelete(path) {
    const res = await
```

```

fetch(`${DB_URL}/${path}.json`, {
  method: "DELETE" });
  if (!res.ok) throw new
Error(`DELETE ${path} failed
(${res.status})`);
  return res.json();
}

// ----- Utilities -----
function uid() { return
Math.random().toString(36).slice(2,
10); }
function todayISO() { return new
Date().toISOString().slice(0, 10); }
function fmtDateLabel(iso) {
  const d = new Date(iso +
"T00:00:00");
  return
d.toLocaleDateString(undefined, {
weekday: "short", month: "short",
day: "numeric" });
}
function fmtTime(t) {
  const [h, m] =
t.split(":").map(Number);
  const period = h >= 12 ? "PM" :
"AM";
  const hr = h % 12 === 0 ? 12 : h %
12;

```

```

    return
    `${hr}:${String(m).padStart(2, "0")}
    ${period}`;
  }
  function durationHours(start, end) {
    const [sh, sm] =
    start.split(":").map(Number);
    const [eh, em] =
    end.split(":").map(Number);
    let mins = (eh * 60 + em) - (sh *
    60 + sm);
    if (mins < 0) mins += 24 * 60;
    return mins / 60;
  }
  function showToast(msg) {
    const t =
    document.getElementById("toast");
    t.textContent = msg;
    t.classList.add("show");
    setTimeout(() =>
    t.classList.remove("show"), 2200);
  }
  function escapeHtml(str) {
    return (str || "").replace(/([<>"'
    "&": "&";", "<": "&lt;";", ">":
    "&gt;";", "'": "&quot;";", '"': "&#39;";"
    ][c]));
  }

```

```

//
=====
=====
// LOAD
//
=====
=====
async function loadAll() {
  try {
    const [blocks, subjects,
practice] = await Promise.all([
      fbGet("studyBlocks"),
      fbGet("subjects"),
      fbGet("practiceLog")
    ]);
    state.blocks = blocks || {};
    state.subjects = subjects || {};
    state.practice = practice || {};
    renderAll();
  } catch (err) {
    console.error(err);
    showToast("Couldn't reach the
database – check your Firebase
rules.");
  }
}

function renderAll() {

```

```

    renderSubjectDropdowns();
    renderDashboard();
    renderBlocksView();
    renderSubjectsView();
    renderPracticeView();
    renderInsights();
}

//
=====
=====
// DASHBOARD
//
=====
=====
function renderDashboard() {

    document.getElementById("todayDate").
    textContent =
        new
        Date().toLocaleDateString(undefined,
        { weekday: "long", year: "numeric",
        month: "long", day: "numeric" });

    const today = todayISO();
    const todaysBlocks =
    Object.entries(state.blocks)
        .filter(([, b]) => b.date ===
    today)

```

```
        .sort((a, b) =>
a[1].start.localeCompare(b[1].start))
;

document.getElementById("statBlocksTo
day").textContent =
todaysBlocks.length;

const hours =
todaysBlocks.reduce((sum, [, b]) =>
sum + durationHours(b.start, b.end),
0);

document.getElementById("statHoursTod
ay").textContent = hours.toFixed(1);

let pending = 0;

Object.values(state.subjects).forEach
(s => {
    Object.values(s.topics ||
{}).forEach(t => { if (!t.completed)
pending++; });
});

document.getElementById("statPending"
).textContent = pending;
```



```

    const listEl =
document.getElementById("todayBlocksList");
    listEl.innerHTML =
todaysBlocks.length
    ? todaysBlocks.map(([id, b]) =>
blockCardHTML(id, b)).join("")
    : `
```

```

Math.floor(streak / 3), 6);
    for (let i = 0; i < flameCount;
i++) {
        const f =
document.createElement("span");
        f.className = "flame";
        f.textContent = "🔥";
        tallyEl.appendChild(f);
    }
}

// quick insight

document.getElementById("dashInsight"
).textContent =
generateInsights().headline;
}

function computeStreak() {
    const days = new
Set(Object.values(state.blocks).map(b
=> b.date));
    let streak = 0;
    let cursor = new Date();
    // if nothing logged today yet,
streak still counts from yesterday
backward
    if (!days.has(todayISO()))
cursor.setDate(cursor.getDate() - 1);

```

```

while (true) {
  const iso =
cursor.toISOString().slice(0, 10);
  if (days.has(iso)) {
    streak++;
    cursor.setDate(cursor.getDate()
- 1);
  } else break;
}
return streak;
}

//
=====
=====
// STUDY BLOCKS
//
=====
=====
function blockCardHTML(id, b) {
  return `
    <div class="block-card" data-
id="${id}">
      <div class="block-card-top">
        <span class="block-
time">${fmtTime(b.start)} -
${fmtTime(b.end)}</span>
        <span class="block-
subject">${escapeHtml(b.subjectName

```

```

|| "General"))</span>
    </div>
    <div class="block-
topic">${escapeHtml(b.topic)}</div>
    ${b.notes ? `<div class="block-
notes">${escapeHtml(b.notes)}</div>`
: ""}
    </div>`;
}

function
attachBlockCardListeners(container) {
    container.querySelectorAll(".block-
card").forEach(card => {
        card.addEventListener("click", ()
=> openBlockModal(card.dataset.id));
    });
}

function renderBlocksView() {
    const entries =
Object.entries(state.blocks).sort((a,
b) => {
        if (a[1].date !== b[1].date)
return
b[1].date.localeCompare(a[1].date);
        return
b[1].start.localeCompare(a[1].start);
    });
}

```

```
    const container =
document.getElementById("allBlocksList");
    if (!entries.length) {
        container.innerHTML = `
```

```
document.getElementById("blockForm");
form.reset();

document.getElementById("deleteBlockB
tn").style.display = "none";

document.getElementById("blockId").va
lue = "";

document.getElementById("blockModalTi
tle").textContent = "New study
block";

    if (id && state.blocks[id]) {
        const b = state.blocks[id];

document.getElementById("blockId").va
lue = id;

document.getElementById("blockDate").
value = b.date;

document.getElementById("blockStart")
.value = b.start;

document.getElementById("blockEnd").v
alue = b.end;

document.getElementById("blockSubject
```

```

    ").value = b.subjectId || "";

    document.getElementById("blockTopic")
    .value = b.topic;

    document.getElementById("blockNotes")
    .value = b.notes || "";

    document.getElementById("blockModalTi
    tle").textContent = "Edit study
    block";

    document.getElementById("deleteBlockB
    tn").style.display = "inline-block";
    } else {

    document.getElementById("blockDate").
    value = todayISO();
    }
    openModal("block");
}

async function saveBlock(e) {
    e.preventDefault();
    const id =
    document.getElementById("blockId").va
    lue;
    const subjectId =
    document.getElementById("blockSubject

```

```

    ).value;
    const subjectName =
state.subjects[subjectId]?.name ||
"General";

    const data = {
      date:
document.getElementById("blockDate").
value,
      start:
document.getElementById("blockStart")
.value,
      end:
document.getElementById("blockEnd").v
alue,
      subjectId,
      subjectName,
      topic:
document.getElementById("blockTopic")
.value.trim(),
      notes:
document.getElementById("blockNotes")
.value.trim(),
      createdAt: id ?
state.blocks[id].createdAt :
Date.now()
    };

    try {

```



```

    if (id) {
      await
      fbPut(`studyBlocks/${id}`, data);
      state.blocks[id] = data;
      showToast("Block updated.");
    } else {
      const res = await
      fbPost("studyBlocks", data);
      state.blocks[res.name] = data;
      showToast("Block logged.");
    }
    closeModal("block");
    renderAll();
  } catch (err) {
    console.error(err);
    showToast("Couldn't save – try
again.");
  }
}

async function deleteBlock() {
  const id =
document.getElementById("blockId").va
lue;
  if (!id) return;
  if (!confirm("Delete this study
block?")) return;
  try {
    await

```

```

fbDelete(`studyBlocks/${id}`);
  delete state.blocks[id];
  closeModal("block");
  renderAll();
  showToast("Block deleted.");
} catch (err) {
  console.error(err);
  showToast("Couldn't delete – try
again.");
}
}

//
=====
=====
// SUBJECTS & TOPICS
//
=====
=====
function renderSubjectDropdowns() {
  const subjectOptions =
Object.entries(state.subjects)
  .sort((a, b) =>
a[1].name.localeCompare(b[1].name))
  .map(([id, s]) => `

```

```

document.getElementById("blockSubject
");
    const practiceSel =
document.getElementById("practiceSubj
ect");
    const filterSel =
document.getElementById("chartSubject
Filter");

    [blockSel, practiceSel].forEach(sel
=> {
        const current = sel.value;
        sel.innerHTML = subjectOptions ||
`<option value="">Add a subject
first</option>`;
        if (current) sel.value = current;
    });

    const currentFilter =
filterSel.value;
    filterSel.innerHTML = `<option
value="all">All subjects</option>` +
subjectOptions;
    filterSel.value = currentFilter ||
"all";
}

function renderSubjectsView() {
    const container =

```

```

document.getElementById("subjectsList
");
    const entries =
Object.entries(state.subjects).sort((
a, b) =>
a[1].name.localeCompare(b[1].name));

    if (!entries.length) {
        container.innerHTML = `
```

```

        <div class="topic-row
        ${t.completed ? "done" : ""}" data-
        subject="${id}" data-topic="${tid}">
            <div class="stamp
            ${t.completed ? "done" : ""}" data-
            action="toggle">${t.completed ? "✓" :
            ""}</div>
                <div class="topic-name"
                data-
                action="edit">${escapeHtml(t.name)}
                </div>
                <button class="topic-edit"
                data-action="edit">edit</button>
            </div>`.join("")
        : `<p class="empty-note">No
        topics yet.</p>`;

    return `
        <div class="subject-card" data-
        id="${id}">
            <div class="subject-head">
                <h3>${escapeHtml(s.name)}
            </h3>
                <span class="subject-
                progress-
                text">${done}/${topics.length}</span>
            </div>
                <div class="subject-bar"><div
                class="subject-bar-fill"

```

```

style="width:${pct}%"></div></div>
    <div
class="topics">${topicsHTML}</div>
    <button class="add-topic-btn"
data-action="add-topic" data-
subject="${id}">+ Add topic</button>
    </div>`;
    }).join("");

    container.querySelectorAll("[data-
action='toggle']").forEach(el => {
        el.addEventListener("click", (e)
=> {
            const row =
e.target.closest(".topic-row");

toggleTopic(row.dataset.subject,
row.dataset.topic);
        });
    });
    container.querySelectorAll("[data-
action='edit']").forEach(el => {
        el.addEventListener("click", (e)
=> {
            const row =
e.target.closest(".topic-row");

openTopicModal(row.dataset.subject,
row.dataset.topic);

```

```

    });
  });
  container.querySelectorAll("[data-
action='add-topic']").forEach(el => {
    el.addEventListener("click", (e)
=>
openTopicModal(e.target.dataset.subje
ct, null));
  });
}

async function toggleTopic(subjectId,
topicId) {
  const t =
state.subjects[subjectId].topics[topi
cId];
  t.completed = !t.completed;
  renderSubjectsView(); // optimistic
  try {
    await
fbPatch(`subjects/${subjectId}/topics
/${topicId}`, { completed:
t.completed });
    renderDashboard();
    renderInsights();
  } catch (err) {
    console.error(err);
    t.completed = !t.completed;
    renderSubjectsView();
  }
}

```

```
        showToast("Couldn't update – try  
again.");  
    }  
}  
  
async function saveSubject(e) {  
    e.preventDefault();  
    const name =  
document.getElementById("subjectName")  
.value.trim();  
    if (!name) return;  
    const data = { name, topics: {} };  
    try {  
        const res = await  
fbPost("subjects", data);  
        const newId = res.name;  
        state.subjects[newId] = data;  
  
document.getElementById("subjectForm")  
.reset();  
        closeModal("subject");  
        renderAll();  
        showToast("Subject added.");  
  
        if (pendingReturn) {  
            const pr = pendingReturn;  
            pendingReturn = null;  
            if (pr.view === "block") {  
                openBlockModal(pr.data.id ||
```



```
null);

document.getElementById("blockDate").
value = pr.data.date;

document.getElementById("blockStart")
.value = pr.data.start;

document.getElementById("blockEnd").v
alue = pr.data.end;

document.getElementById("blockTopic")
.value = pr.data.topic;

document.getElementById("blockNotes")
.value = pr.data.notes;

document.getElementById("blockSubject
").value = newId;
    } else if (pr.view ===
"practice") {

document.getElementById("practiceForm
").reset();
    openModal("practice");

document.getElementById("practiceDate
").value = pr.data.date;
```

```
document.getElementById("practiceAttempted").value = pr.data.attempted;

document.getElementById("practiceCorrect").value = pr.data.correct;

document.getElementById("practiceSubject").value = newId;
    }
}
} catch (err) {
    console.error(err);
    showToast("Couldn't save – try again.");
}
}

function openTopicModal(subjectId,
topicId) {
    const form =
document.getElementById("topicForm");
    form.reset();

document.getElementById("topicSubjectId").value = subjectId;

document.getElementById("topicId").value = topicId || "";
```

```
document.getElementById("deleteTopicB
tn").style.display = topicId ?
"inline-block" : "none";

document.getElementById("topicModalTi
tle").textContent = topicId ? "Edit
topic" : "New topic";
    if (topicId) {

document.getElementById("topicName").
value =
state.subjects[subjectId].topics[topi
cId].name;
    }
    openModal("topic");
}

async function saveTopic(e) {
    e.preventDefault();
    const subjectId =
document.getElementById("topicSubject
Id").value;
    const topicId =
document.getElementById("topicId").va
lue;
    const name =
document.getElementById("topicName").
value.trim();
    if (!name) return;
```

```
    try {
      if (topicId) {
        await
        fbPatch(`subjects/${subjectId}/topics/${topicId}`, { name });

        state.subjects[subjectId].topics[topicId].name = name;
        showToast("Topic updated.");
      } else {
        const data = { name, completed: false, createdAt: Date.now() };
        const res = await
        fbPost(`subjects/${subjectId}/topics`, data);
        if
        (!state.subjects[subjectId].topics)
        state.subjects[subjectId].topics = {};

        state.subjects[subjectId].topics[res.name] = data;
        showToast("Topic added.");
      }
      closeModal("topic");
      renderAll();
    } catch (err) {
      console.error(err);
    }
  }
}
```

```
        showToast("Couldn't save – try  
again.");  
    }  
}  
  
async function deleteTopic() {  
    const subjectId =  
document.getElementById("topicSubject  
Id").value;  
    const topicId =  
document.getElementById("topicId").va  
lue;  
    if (!topicId) return;  
    if (!confirm("Delete this topic?"))  
return;  
    try {  
        await  
fbDelete(`subjects/${subjectId}/topic  
s/${topicId}`);  
        delete  
state.subjects[subjectId].topics[topi  
cId];  
        closeModal("topic");  
        renderAll();  
        showToast("Topic deleted.");  
    } catch (err) {  
        console.error(err);  
        showToast("Couldn't delete – try  
again.");  
    }  
}
```

```

    }
  }

  //
  =====
  =====
  // PRACTICE LOG / CHART
  //
  =====
  =====
  function renderPracticeView() {
    const filter =
document.getElementById("chartSubject
Filter").value || "all";
    const entries =
Object.entries(state.practice)
      .filter(([, p]) => filter ===
"all" || p.subjectId === filter)
      .sort((a, b) =>
a[1].date.localeCompare(b[1].date));

    // list (most recent first)
    const listEl =
document.getElementById("practiceLogL
ist");
    const reversed =
[...entries].reverse();
    listEl.innerHTML = reversed.length
      ? reversed.map(([id, p]) => `

```

```

        <div class="practice-row">
            <div class="practice-meta">
                <span
class="pdate">${fmtDateLabel(p.date)}
                </span>

                <span>${escapeHtml(p.subjectName)}
                </span>
            </div>
            <div class="practice-
stats">${p.correct}/${p.attempted}
correct (${Math.round((p.correct /
p.attempted) * 100 || 0)}%)</div>
            </div>`).join("")
        : `<p class="empty-note">No
practice sessions logged yet.</p>`;

    // chart
    const labels = entries.map(([, p])
=> p.date);
    const attempted = entries.map(([,
p]) => p.attempted);
    const accuracy = entries.map(([,
p]) => Math.round((p.correct /
p.attempted) * 100 || 0));

    const ctx =
document.getElementById("progressChar
t");

```

```

if (chart) chart.destroy();
chart = new Chart(ctx, {
  data: {
    labels:
labels.map(fmtDateLabel),
    datasets: [
      {
        type: "bar",
        label: "Questions
attempted",
        data: attempted,
        backgroundColor:
"#8C6FF755",
        borderColor: "#8C6FF7",
        borderWidth: 1,
        yAxisID: "y",
        borderRadius: 6
      },
      {
        type: "line",
        label: "Accuracy %",
        data: accuracy,
        borderColor: "#FF6FA5",
        backgroundColor: "#FF6FA5",
        tension: 0.35,
        yAxisID: "y1",
        pointRadius: 4,
        pointBackgroundColor:
"#FF6FA5"

```



```

    }
  ]
},
options: {
  responsive: true,
  interaction: { mode: "index",
intersect: false },
  scales: {
    y: { beginAtZero: true,
position: "left", title: { display:
true, text: "Questions" } },
    y1: { beginAtZero: true, max:
100, position: "right", grid: {
drawOnChartArea: false }, title: {
display: true, text: "Accuracy %" } }
  },
  plugins: { legend: { position:
"bottom" } }
}
});
}

```

```

async function savePractice(e) {
  e.preventDefault();
  const subjectId =
document.getElementById("practiceSubj
ect").value;
  const subjectName =
state.subjects[subjectId]?.name ||

```

```
"General";
  const data = {
    date:
document.getElementById("practiceDate
").value,
    subjectId,
    subjectName,
    attempted:
Number(document.getElementById("pract
iceAttempted").value),
    correct:
Number(document.getElementById("pract
iceCorrect").value),
    createdAt: Date.now()
  };
  try {
    const res = await
fbPost("practiceLog", data);
    state.practice[res.name] = data;

document.getElementById("practiceForm
").reset();
    closeModal("practice");
    renderAll();
    showToast("Session logged.");
  } catch (err) {
    console.error(err);
    showToast("Couldn't save – try
again.");
```

```

    }
  }

  //
  =====
  =====
  // INSIGHTS ENGINE (rule-based
  analysis over your own logged data)
  //
  =====
  =====
  function generateInsights() {
    const blocks =
Object.values(state.blocks);
    const practice =
Object.values(state.practice);
    const cards = [];

    if (!blocks.length) {
      return { headline: "Log a few
study blocks and I'll start reading
your patterns.", cards: [] };
    }

    const streak = computeStreak();
    const days = [...new
Set(blocks.map(b => b.date))].sort();
    const last14 = days.filter(d =>
(Date.now() - new Date(d).getTime())

```

```
/ 86400000 <= 14);

// --- streak read ---
let headline = "";
if (streak === 0) {
    headline = "No active streak –
log a block today to start one.";
    cards.push({ tone: "warn", title:
"Streak broken", body: "You don't
have a session logged today or
yesterday yet. One block today
restarts the count – the hardest part
is usually just opening this page."
});
} else if (streak < 3) {
    headline = `${streak}-day streak
– early days, keep it going.`;
    cards.push({ tone: "good", title:
"Streak building", body: `${streak}
day${streak > 1 ? "s" : ""} in a row.
The first week is the hardest part of
building the habit – don't break it
for something that can wait.` });
} else {
    headline = `${streak}-day streak
– this is becoming a habit.`;
    cards.push({ tone: "good", title:
"Streak holding", body: `${streak}
consecutive days logged. That's well
```

```

past the point where it's just
willpower – it's routine now.` });
    }

    // --- subject balance ---
    const subjectHours = {};
    blocks.forEach(b => {
        const key = b.subjectName ||
"General";
        subjectHours[key] =
(subjectHours[key] || 0) +
durationHours(b.start, b.end);
    });
    const sorted =
Object.entries(subjectHours).sort((a,
b) => b[1] - a[1]);
    if (sorted.length > 1) {
        const [topSubj, topHrs] =
sorted[0];
        const [lowSubj, lowHrs] =
sorted[sorted.length - 1];
        if (topHrs > lowHrs * 2.5 &&
topHrs - lowHrs > 2) {
            cards.push({
                tone: "warn",
                title: "Uneven split",
                body: `${topSubj} has
${topHrs.toFixed(1)}h logged versus
${lowHrs.toFixed(1)}h on ${lowSubj}.

```

```

Worth a deliberate block on
${lowSubj} soon so it doesn't fall
further behind.`
    });
  }
}

// --- pending topics per subject -
--
const neglected = [];

Object.values(state.subjects).forEach
(s => {
  const topics =
Object.values(s.topics || {});
  const pending = topics.filter(t
=> !t.completed).length;
  if (topics.length && pending /
topics.length > 0.6 && pending >= 3)
  {
    neglected.push([s.name,
pending, topics.length]);
  }
});
if (neglected.length) {
  const [name, pending, total] =
neglected.sort((a, b) => b[1] - a[1])
[0];
  cards.push({

```

```

        tone: "warn",
        title: "Backlog forming",
        body: `${name} has ${pending}
of ${total} topics still open.
Consider a dedicated block just for
clearing 2-3 of them rather than
adding new ones.`
    });
}

// --- gaps in last 14 days ---
if (days.length >= 3) {
    const gapDays = 14 -
last14.length;
    if (gapDays >= 6) {
        cards.push({
            tone: "warn",
            title: "Quiet stretch",
            body: `Only ${last14.length}
of the last 14 days have a logged
block. Consistency compounds –
shorter, more frequent blocks tend to
stick better than occasional long
ones.`
        });
    }
}

// --- practice accuracy trend ---

```

```
    if (practice.length >= 3) {
      const sortedPractice =
[...practice].sort((a, b) =>
a.date.localeCompare(b.date));
      const half =
Math.floor(sortedPractice.length /
2);
      const firstHalf =
sortedPractice.slice(0, half);
      const secondHalf =
sortedPractice.slice(half);
      const acc = arr => {
        const a = arr.reduce((s, p) =>
s + p.attempted, 0);
        const c = arr.reduce((s, p) =>
s + p.correct, 0);
        return a ? (c / a) * 100 : 0;
      };
      const accFirst = acc(firstHalf);
      const accSecond =
acc(secondHalf);
      const delta = accSecond -
accFirst;
      if (Math.abs(delta) >= 5) {
        cards.push({
          tone: delta > 0 ? "good" :
"warn",
          title: delta > 0 ? "Accuracy
improving" : "Accuracy dipping",
```



```

        body: `Practice accuracy has
moved from ~${accFirst.toFixed(0)}%
to ~${accSecond.toFixed(0)}% across
your logged sessions. ${delta > 0 ?
"Whatever changed recently is working
– keep the same approach." : "Might
be worth slowing down or revisiting
fundamentals before adding more
volume."}`
    });
}
} else if (!practice.length) {
  cards.push({
    tone: "warn",
    title: "No practice logged",
    body: "You haven't logged any
practice sessions yet. Study blocks
build understanding, but question
practice is what the graph – and your
exams – actually measure."
  });
}

// --- avg session length ---
const avgHrs = blocks.reduce((s, b)
=> s + durationHours(b.start, b.end),
0) / blocks.length;
if (avgHrs > 3) {
  cards.push({

```

```

        tone: "warn",
        title: "Long sessions",
        body: `Average block is
        ${avgHrs.toFixed(1)}h. Marathon
        sessions are easy to skip on a bad
        day – shorter, more frequent blocks
        are usually easier to stay consistent
        with.`
    });
}

    if (!cards.length) {
        cards.push({ tone: "good", title:
        "Steady", body: "Nothing concerning
        in the data right now – subjects are
        reasonably balanced and sessions are
        being logged regularly." });
    }

    return { headline, cards };
}

function renderInsights() {
    const { cards } =
    generateInsights();
    const container =
    document.getElementById("insightsFull
    ");
    container.innerHTML = cards.map(c

```

```

=> `
    <div class="insight-card
    ${c.tone}">
        <h4>${escapeHtml(c.title)}</h4>
        <p
        style="margin:0">${escapeHtml(c.body)
        }</p>
        </div>`).join("");
    }

    //
    =====
    =====
    // MODALS & NAV
    //
    =====
    =====
    function openModal(name) {
    document.getElementById(`modal-${name}
    `).classList.add("open"); }
    function closeModal(name) {
    document.getElementById(`modal-${name}
    `).classList.remove("open"); }

    function initNav() {
        document.querySelectorAll(".rail-
        btn").forEach(btn => {
            btn.addEventListener("click", ()
            => {

```

```

document.querySelectorAll(".rail-
btn").forEach(b =>
b.classList.remove("active"));

document.querySelectorAll(".view").fo
rEach(v =>
v.classList.remove("active"));
    btn.classList.add("active");

document.getElementById(`view-${btn.d
ataset.view}`).classList.add("active"
);
    });
    });
}

function initModals() {
    document.querySelectorAll("[data-
open-modal]").forEach(btn => {
        btn.addEventListener("click", ()
=> {
            const name =
btn.dataset.openModal;
            if (name === "block")
openBlockModal(null);
            else if (name === "practice") {

document.getElementById("practiceForm

```

```

").reset();

document.getElementById("practiceDate")
.value = todayISO();
    openModal("practice");
    } else openModal(name);
    });
    });
    document.querySelectorAll("[data-
quick-add-subject]").forEach(btn => {
    btn.addEventListener("click", ()
=> {
        const returnTo =
btn.dataset.quickAddSubject;
        if (returnTo === "block") {
            pendingReturn = {
                view: "block",
                data: {
                    id:
document.getElementById("blockId").va
lue,
                    date:
document.getElementById("blockDate").
value,
                    start:
document.getElementById("blockStart")
.value,
                    end:
document.getElementById("blockEnd").v

```

```

    alue,
        topic:
document.getElementById("blockTopic")
.value,
        notes:
document.getElementById("blockNotes")
.value
    }
};
closeModal("block");
} else if (returnTo ===
"practice") {
    pendingReturn = {
        view: "practice",
        data: {
            date:
document.getElementById("practiceDate
").value,
            attempted:
document.getElementById("practiceAtte
mpted").value,
            correct:
document.getElementById("practiceCorr
ect").value
        }
    };
    closeModal("practice");
}

```

```
document.getElementById("subjectForm")
).reset();
    openModal("subject");
    });
});
document.querySelectorAll("[data-
close-modal]").forEach(btn => {
    btn.addEventListener("click", ()
=> btn.closest(".modal-
overlay").classList.remove("open"));
});
document.querySelectorAll(".modal-
overlay").forEach(ov => {
    ov.addEventListener("click", (e)
=> { if (e.target === ov)
ov.classList.remove("open"); });
});
}

function initForms() {

document.getElementById("blockForm").
addEventListener("submit",
saveBlock);

document.getElementById("deleteBlockB
tn").addEventListener("click",
deleteBlock);
```

```
document.getElementById("subjectForm")
).addEventListener("submit",
saveSubject);
```

```
document.getElementById("topicForm").
addEventListener("submit",
saveTopic);
```

```
document.getElementById("deleteTopicB
tn").addEventListener("click",
deleteTopic);
```

```
document.getElementById("practiceForm
").addEventListener("submit",
savePractice);
```

```
document.getElementById("chartSubject
Filter").addEventListener("change",
renderPracticeView);
```

```
document.getElementById("refreshInsig
hts").addEventListener("click", () =>
{ renderInsights(); showToast("Re-
read."); });
}
```

```
// ----- Boot -----
document.addEventListener("DOMContentLoaded", () => {
```



```
    initNav();  
    initModals();  
    initForms();  
    loadAll();  
});
```